

Part 1

```
PS C:\Users\szhan\paralcomp> javac pr_task1.java
PS C:\Users\szhan\paralcomp> java pr_task1
Run 1: pi ~= 2.588091 (totalHits=32351133, time=2087ms)
Run 2: pi ~= 2.867713 (totalHits=35846414, time=2261ms)
Run 3: pi ~= 2.907766 (totalHits=36347075, time=2256ms)
Run 4: pi ~= 2.921454 (totalHits=36518174, time=2302ms)
Run 5: pi ~= 2.918888 (totalHits=36486105, time=2259ms)
PS C:\Users\szhan\paralcomp> █
```

Part 2

```
PS C:\Users\szhan\paralcomp> javac pr_task2.java
>> java pr_task2
=== Single-threaded baseline ===
Run 1: pi ~= 3.141376 (hits=39267199, time=3586ms)
Run 2: pi ~= 3.141973 (hits=39274666, time=3553ms)
Run 3: pi ~= 3.141549 (hits=39269362, time=4412ms)

=== 4-thread, AtomicLong (race fixed, lock-free) ===
Run 1: pi ~= 3.141323 (totalHits=39266542, time=2090ms)
Run 2: pi ~= 3.141743 (totalHits=39271785, time=2043ms)
Run 3: pi ~= 3.141395 (totalHits=39267435, time=2011ms)

=== 4-thread, synchronized (race fixed, lock-based) ===
Run 1: pi ~= 3.141782 (syncHits=39272271, time=8014ms)
Run 2: pi ~= 3.141351 (syncHits=39266883, time=8024ms)
Run 3: pi ~= 3.141317 (syncHits=39266463, time=7792ms)
PS C:\Users\szhan\paralcomp> █
```

Part 3

```
PS C:\Users\szhan\paralcomp> javac pr_task3.java
>> java pr_task3
```

T	Time(ms)	Speedup (T1/TN)	Efficiency
1	6833	1.00	x 100.0 %
2	3016	2.27	x 113.3 %
4	1511	4.52	x 113.1 %
8	843	8.11	x 101.3 %
16	617	11.07	x 69.2 %
32	656	10.42	x 32.6 %

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

Because the CPU has a fixed number of physical cores (8, based on where efficiency stays near 100%), and hyper-threading only provides logical cores that share the same physical execution resources (ALUs, caches, pipelines) with a real core -- they don't add genuine extra parallel compute capacity. Once thread count exceeds the number of physical cores, additional threads are either sharing a physical core's resources (mild benefit, as seen going 8->16) or directly competing with already-running threads for CPU time (net loss, as seen going 16->32). Doubling the thread count only doubles real throughput when there is genuinely double the hardware to run them on -- past that point, you're not adding compute, you're adding contention.

2. Why was the synchronized version in Part 2 slower than running on one single core?

Because `synchronized` serializes access to `syncHits` through a monitor lock, and with 4 threads calling `incrementSyncHits()` a combined ~39 million times, most of the runtime goes into lock acquisition/release and keeping the counter's value visible across CPU cores (cache-line invalidation/memory fences), rather than into the actual Monte Carlo math. This lock overhead is large enough that it outweighs

not just the benefit of parallelism, but even the cost of running everything on a single thread with no locking at all.

By contrast, `AtomicLong` avoids this because CAS is a much cheaper, lock-free hardware operation — threads never block, they just retry — so the parallel speedup from splitting the random-number generation across 4 cores isn't eaten up by synchronization overhead.